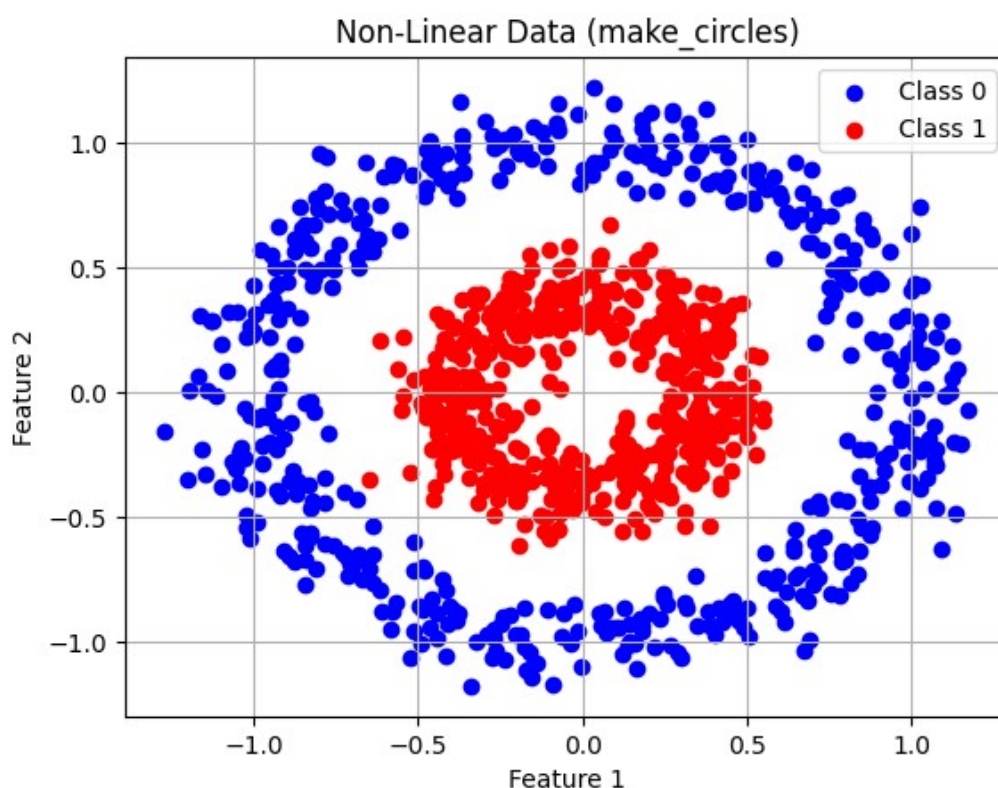


✓ Day 28: Non-Linear Binary Classification in TensorFlow

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
from sklearn.datasets import make_circles

# Generating non-linear data using make_circles
X_train, y_train = make_circles(n_samples=1000, noise=0.1, factor=0.4)

# Visualizing the data
plt.scatter(X_train[y_train == 0][:, 0], X_train[y_train == 0][:, 1], color='blue', label='Class 0')
plt.scatter(X_train[y_train == 1][:, 0], X_train[y_train == 1][:, 1], color='red', label='Class 1')
plt.title('Non-Linear Data (make_circles)')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.legend()
plt.grid(True)
plt.show()
```



```
# Building a non-linear model with hidden layers
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(2,)), # Hidden layer 1
    tf.keras.layers.Dense(32, activation='relu'), # Hidden layer 2
    tf.keras.layers.Dense(1, activation='sigmoid') # Output layer
])
```

```
# Compiling the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'],)
```

```
⚡ /usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
# Training the model
history = model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=1)
```

```
# Evaluating the accuracy on the training data
loss, accuracy = model.evaluate(X_train, y_train)
print(f'Final Training Accuracy: {accuracy * 100:.2f}%')
```

```
⚡ Epoch 1/50
32/32 ————— 2s 2ms/step - accuracy: 0.5001 - loss: 0.6993
Epoch 2/50
32/32 ————— 0s 2ms/step - accuracy: 0.5148 - loss: 0.6580
Epoch 3/50
32/32 ————— 0s 2ms/step - accuracy: 0.5497 - loss: 0.6214
Epoch 4/50
32/32 ————— 0s 2ms/step - accuracy: 0.6720 - loss: 0.5849
Epoch 5/50
32/32 ————— 0s 1ms/step - accuracy: 0.8179 - loss: 0.5207
Epoch 6/50
32/32 ————— 0s 2ms/step - accuracy: 0.9243 - loss: 0.4467
Epoch 7/50
32/32 ————— 0s 2ms/step - accuracy: 0.9897 - loss: 0.3649
Epoch 8/50
32/32 ————— 0s 2ms/step - accuracy: 0.9878 - loss: 0.2898
Epoch 9/50
32/32 ————— 0s 2ms/step - accuracy: 0.9957 - loss: 0.2249
Epoch 10/50
32/32 ————— 0s 2ms/step - accuracy: 0.9978 - loss: 0.1678
Epoch 11/50
32/32 ————— 0s 2ms/step - accuracy: 0.9997 - loss: 0.1297
Epoch 12/50
32/32 ————— 0s 2ms/step - accuracy: 0.9995 - loss: 0.1070
Epoch 13/50
32/32 ————— 0s 2ms/step - accuracy: 0.9994 - loss: 0.0845
Epoch 14/50
32/32 ————— 0s 2ms/step - accuracy: 0.9981 - loss: 0.0676
Epoch 15/50
32/32 ————— 0s 2ms/step - accuracy: 0.9994 - loss: 0.0564
Epoch 16/50
32/32 ————— 0s 2ms/step - accuracy: 0.9981 - loss: 0.0502
Epoch 17/50
32/32 ————— 0s 2ms/step - accuracy: 0.9984 - loss: 0.0426
Epoch 18/50
32/32 ————— 0s 2ms/step - accuracy: 0.9998 - loss: 0.0378
Epoch 19/50
32/32 ————— 0s 2ms/step - accuracy: 0.9998 - loss: 0.0318
Epoch 20/50
32/32 ————— 0s 2ms/step - accuracy: 0.9998 - loss: 0.0296
Epoch 21/50
32/32 ————— 0s 2ms/step - accuracy: 0.9989 - loss: 0.0275
Epoch 22/50
32/32 ————— 0s 2ms/step - accuracy: 0.9995 - loss: 0.0238
Epoch 23/50
```

```

Epoch 23/50
32/32 ————— 0s 2ms/step - accuracy: 0.9988 - loss: 0.0226
Epoch 24/50
32/32 ————— 0s 2ms/step - accuracy: 0.9961 - loss: 0.0223
Epoch 25/50
32/32 ————— 0s 2ms/step - accuracy: 0.9984 - loss: 0.0176
Epoch 26/50
32/32 ————— 0s 2ms/step - accuracy: 0.9995 - loss: 0.0157
Epoch 27/50
32/32 ————— 0s 2ms/step - accuracy: 0.9975 - loss: 0.0178
Epoch 28/50
32/32 ————— 0s 2ms/step - accuracy: 0.9998 - loss: 0.0126
Epoch 29/50
32/32 ————— 0s 2ms/step - accuracy: 0.9997 - loss: 0.0118

```

```

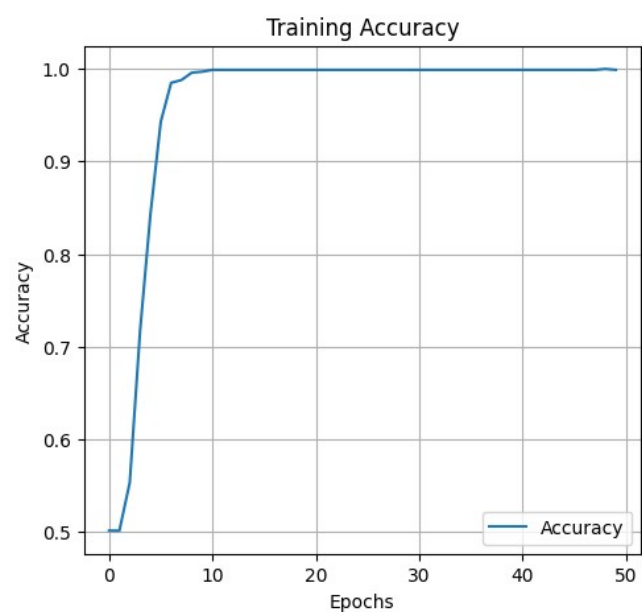
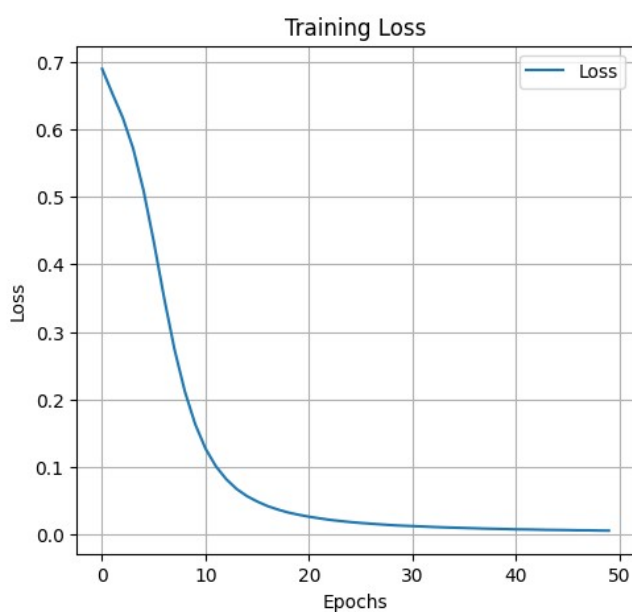
# Plotting the training loss and accuracy
plt.figure(figsize=(12, 5))

# Loss plot
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Loss')
plt.title('Training Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

# Accuracy plot
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'], label='Accuracy')
plt.title('Training Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.show()

```

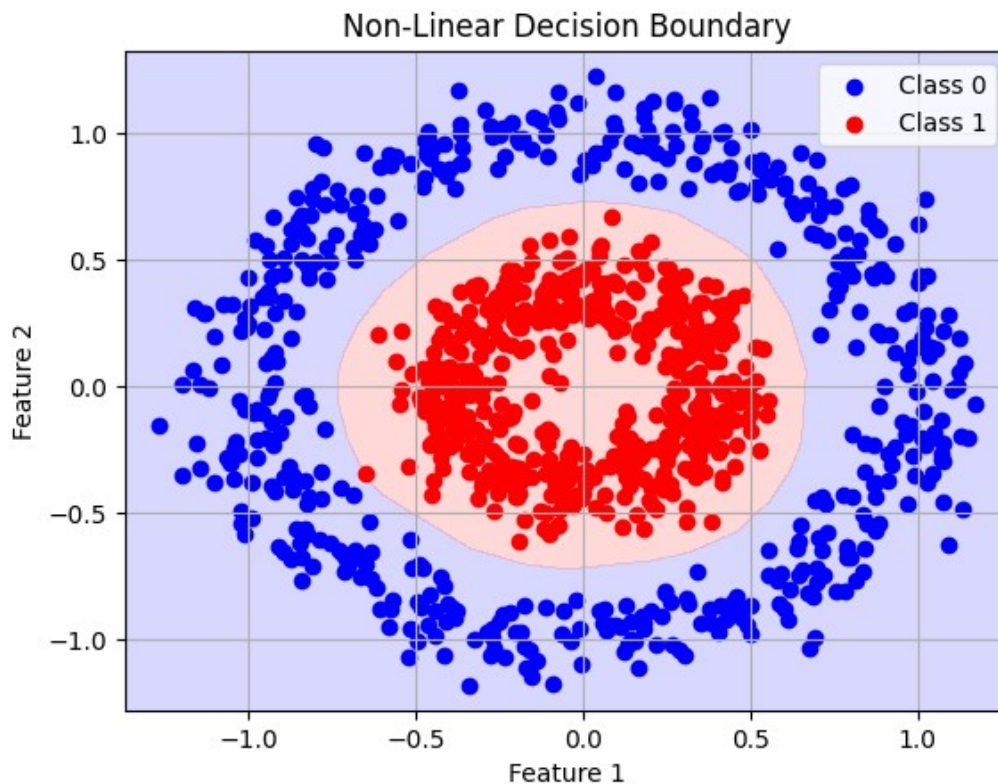


```
# Function to plot decision boundary
def plot_decision_boundary(model, X, y):
    x_min, x_max = X[:, 0].min() - 0.1, X[:, 0].max() + 0.1
    y_min, y_max = X[:, 1].min() - 0.1, X[:, 1].max() + 0.1
    xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.01),
                          np.arange(y_min, y_max, 0.01))
    grid = np.c_[xx.ravel(), yy.ravel()]
    pred = model.predict(grid)
    pred = pred.reshape(xx.shape)

    plt.contourf(xx, yy, pred, levels=[0, 0.5, 1], cmap='bwr', alpha=0.3)
    plt.scatter(X[y == 0][:, 0], X[y == 0][:, 1], color='blue', label='Class 0')
    plt.scatter(X[y == 1][:, 0], X[y == 1][:, 1], color='red', label='Class 1')
    plt.title('Non-Linear Decision Boundary')
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.legend()
    plt.grid(True)
    plt.show()

# Plotting the decision boundary
plot_decision_boundary(model, X_train, y_train)
```

2162/2162 ————— 7s 3ms/step



```
# Increasing the model complexity for better accuracy
model_optimized = tf.keras.Sequential([
    tf.keras.layers.Dense(32, activation='relu', input_shape=(2,)), # Increased nodes in hidden
```

```
tf.keras.layers.Dense(64, activation='relu'),
tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compiling the optimized model
model_optimized.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Training the optimized model
history_optimized = model_optimized.fit(X_train, y_train, epochs=100, batch_size=32, verbose=0)

# Evaluating the optimized model accuracy
loss, accuracy = model_optimized.evaluate(X_train, y_train)
print(f'Optimized Model Accuracy: {accuracy * 100:.2f}%')
```

 32/32  0s 1ms/step - accuracy: 1.0000 - loss: 6.1376e-04
Optimized Model Accuracy: 100.00%

"Non-linear binary classification allows models to create more complex decision boundaries, essential for data that isn't linearly separable. With optimized architectures, achieving accuracies above 99% is possible, demonstrating the power of deep learning in handling complex tasks.